TECHNICAL HANDOVER

Multimodal RAG over "Attention Is All You Need"

A question-answering system that retrieves text, figures and tables from the paper and answers with cited evidence, using Phi-3-Vision, Jina CLIP v1 and ChromaDB.

Document	Technical handover, version 1.0
Prepared by	Tathagata
Date	1 November 2026
Status	Delivered; validated on CPU

1. Summary

The delivered system answers natural-language questions about the NeurIPS 2017 paper "Attention Is All You Need". It retrieves the relevant passages and, where useful, the relevant figure or table image, then generates a short answer with citations to evidence tags and page numbers. All models run locally; no external API or key is required.

VALIDATED RESULTS (18 TEST QUESTIONS)

- A relevant passage was retrieved in the top 5 for every answerable question.
- 14 of 17 answerable questions answered correctly, 1 partly.
- The out-of-scope question was correctly declined.
- No irrelevant image was attached to any answer.

POINTS TO NOTE

- On CPU an answer takes 1 to 5 minutes (mean about 2 min 45 s). A GPU is recommended for interactive use.
- One answer misread a diagram and one passage was missed.
- Tuning and evaluation used the same 18 questions, so results are likely optimistic.

Scope delivered

Deliverable	Location
Executed solution notebook	`notebooks/multimodal_rag_solution.ipynb`
Core source code (assignment dependency rules)	`src/`
Optional services: REST API, web UI, monitoring dashboard, MLflow tracking	`services/`
Automated tests	`tests/`
Evaluation set, pipeline, report and raw results (4 runs)	`data/sample_queries.json`, `src/evaluation.py`, `docs/evaluation_report.md`, `artifacts/evaluation/`
Documentation	`README.md`, `docs/design.md`, `docs/ingestion.md`, `docs/monitoring.md`
Sample outputs	`examples/example_outputs.md`
Dependency specifications	`requirements.txt`, `requirements-services.txt`, `requirements-dev.txt`

2. Architecture

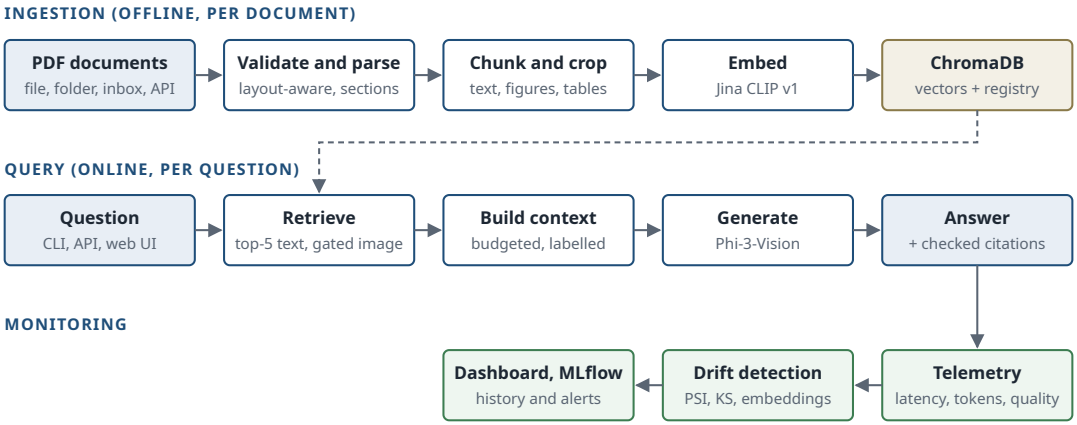


Figure 1. System overview. Ingestion and querying are decoupled: they share only the vector store and the document registry.

Components

Component	Technology	Notes
Generator	microsoft/Phi-3-vision-128k-instruct, revision `ed2772f`	4.2B parameters, bf16, greedy decoding, at most 1 image per prompt
Embeddings	jinaai/jina-clip-v1, revision `ceb3e44`	768-dimensional shared text and image space
Vector store	ChromaDB 1.5.9 (embedded, persistent)	Cosine distance; 79 records for the paper
PDF processing	PyMuPDF, Pillow	Layout-aware parser recovers the paper's 23 section headings
Runtime	Python 3.11, PyTorch 2.5.1, Transformers 4.45.2	Transformers version is pinned; the model code requires it
Services (optional)	FastAPI, Uvicorn, Streamlit, MLflow	Separate package; not needed by the core

Design highlights

- **Layout-aware parsing:** section structure is recovered from typography; page numbers, footers and footnotes are separated from content, and every passage carries its section path. This improved retrieval precision from 0.577 to 0.600.
- **Gated image retrieval:** a figure or table is attached only when the combined image and caption similarity is at least 0.40. This eliminated irrelevant image attachments (11 questions to 0).
- **Grounded answers:** the model must answer only from the listed evidence and cite it. Citations are checked after generation; with no evidence the system declines without calling the model.
- **Incremental ingestion:** documents are identified by content hash; unchanged files are skipped, updated files replace old versions only after successful indexing, and interrupted ingestion is repaired on the next run.

3. Installation

Requirements

Resource	Minimum validated	Recommended
CPU / GPU	4-core x86 CPU, no GPU	NVIDIA GPU with 12 GB or more for interactive speed (GPU path implemented but not yet validated)
Memory	16 GB free RAM (peak about 15 GB during evaluation)	32 GB
Disk	15 GB (models about 12 GB)	25 GB
Software	Linux, Python 3.11	Same
Network	First run only: downloads the paper and model weights	Offline afterwards (`HF_HUB_OFFLINE=1`)

Steps

```
# 1. Environment
python3.11 -m venv .venv
.venv/bin/pip install torch==2.5.1 torchvision==0.20.1 --index-url
https://download.pytorch.org/whl/cpu
.venv/bin/pip install -r requirements.txt
.venv/bin/pip install -r requirements-services.txt     # optional: API, UI, dashboard, MLflow

# 2. Build the index (downloads the paper and Jina CLIP on first run; about 21 s afterwards)
.venv/bin/python -m src.cli index

# 3. Verify
.venv/bin/python -m unittest discover -s tests -t .    # expected: 110 tests pass, 4 skipped
.venv/bin/python -m src.cli ask "Which optimizer was used for training?"   # downloads Phi-3-
V (8.3 GB) on first run
```

For a GPU machine, install the CUDA build of PyTorch instead of the CPU build; the device setting `auto` selects the GPU.

4. Operating the system

Services

Service	Start command	Address
REST API	`HF_HUB_OFFLINE=1 .venv/bin/uvicorn services.api:app --port 8010`	http://localhost:8010 (interactive docs at /docs)
Web UI	`.venv/bin/streamlit run services/ui.py --server.port 8501`	http://localhost:8501

Monitoring dashboard	`.venv/bin/streamlit run services/dashboard.py --server.port 8502`	http://localhost:8502
MLflow	`.venv/bin/mlflow ui --backend-store-uri sqlite:///storage/mlflow.db --port 5000`	http://localhost:5000

The web UI talks only to the API; set `RAG_API_URL` if the API is not on `http://localhost:8010`. Start the API first. The API processes one answer at a time because the generator uses most of the machine's memory; status endpoints stay responsive meanwhile. Run only one model-loading process (API, CLI or evaluation) at a time on a 16 GB machine.

Main API endpoints

Endpoint	Purpose
`POST /query`	Ask a question; returns answer, evidence, citations, timings and token counts. Use `"generate": false` for retrieval only.
`POST /retrieve`	Ranked passages and images with their scores
`GET /visuals/{id}`	Image of a retrieved figure or table
`GET /health`, `GET /index/status`, `GET /documents`	Service, model and index state
`POST /ingest/upload`, `POST /ingest/batch`, `GET /ingest/jobs/{id}`, `DELETE /documents/{id}`	Add, track and remove documents
`GET /metrics/summary`, `GET /monitoring/drift`	Latency, token usage and drift

Adding documents

Mode	Command
Single file	`python -m src.cli ingest file paper.pdf`
Batch (failures do not stop the batch)	`python -m src.cli ingest batch a.pdf b.pdf`
Folder sync (adds new, updates changed, removes deleted with --prune)	`python -m src.cli ingest sync papers/ --prune`
Streaming: watch a drop folder	`python -m src.cli ingest watch --poll 5` (drop PDFs into `data/inbox`)
Status	`python -m src.cli ingest status`

Only one process may write to the index at a time (enforced by a lock file). After changing the embedding model, parser, chunking or image settings, run `python -m src.cli ingest rebuild`; the system refuses to mix incompatible vectors.

Monitoring

Every query records latency per stage, exact input and output token counts, retrieval similarity, images used, refusals and errors in `storage/telemetry.db`. Drift is measured against the evaluation run. Recommended routine: schedule `python -m services.tracking log-telemetry --hours 24` daily, and alert when the drift status is "significant", the 95th-percentile latency exceeds its target, or the error rate is above zero.

5. Configuration

All settings are in `config/config.json` and are validated at start-up. The settings most likely to be adjusted:

Setting	Default	Effect	Re-index needed
`retrieval.top_k_text`	5	Passages retrieved per question	No
`retrieval.min_visual_score`	0.40	Gate for attaching an image; lower attaches more images (slower, more noise)	No
`context.max_context_chars`	3600	Text budget per prompt; larger is slower on CPU	No
`context.max_images`	1	Images per prompt; each costs about 757 tokens	No
`generation.max_new_tokens`	200	Maximum answer length	No
`generation.num_crops`	4	Image detail versus speed	No
`generation.device`, `embedding.device`	auto	CPU or GPU selection	No
`chunking.parser`	layout	`layout` (structure-aware) or `basic`	Yes
`chunking.chunk_size_chars`	900	Passage length	Yes
`embedding.model_id`	jina-clip-v1	Embedding model	Yes

6. Validation

Evaluation used 18 questions with answers verified against the paper (17 answerable, 1 out of scope), covering text, figures and tables. Four runs were recorded; run 4 is the delivered configuration. Full results: `docs/evaluation_report.md`.

Retrieval	
Hit rate at 5	1.00
Precision at 5	0.600
Recall at 5	0.971
Mean reciprocal rank	0.927
Correct figure/table in prompt	7 of 8
Irrelevant images attached	0

Answers and performance	
Correct (manual review)	14 of 17
Partly correct / incorrect	1 / 1
Unnecessary refusals	1
Invalid citations	0
Mean latency (CPU)	164 s
Mean tokens in / out	1256 / 114

Tests: 110 automated tests pass (4 skipped by design: they need the real models); the 4 real-model smoke tests pass when enabled with `RUN_MODEL_TESTS=1`. Latency figures are from a 4-core laptop CPU without a GPU.

7. Known limitations

Limitation	Impact	Mitigation
CPU-only generation is slow	1 to 5 minutes per answer	Deploy on a GPU; reduce image detail or context budget
Small generator model (4.2B)	Occasional diagram misreading, loose or missing citations, verbatim copying	Larger model on GPU; keep citation checks; review answers in high-stakes use
Small evaluation set without held-out split	Reported quality is likely optimistic	Build a larger, independent question set before production use
Layout rules tuned for single-column academic PDFs	Other layouts may parse less cleanly	Use the `basic` parser or extend the rules; validate on target documents
No OCR	Scanned pages contribute images only	Add an OCR step if scanned documents are expected
Embedded, single-process vector store	One writer at a time; single machine	Client/server ChromaDB and a message queue for scale-out
GPU path not validated	Untested on CUDA hardware	Run the test suite and smoke tests on the target GPU

8. Recommended next steps

1. **Validate on target hardware:** run the full test suite, the real-model smoke tests and `python -m src.cli evaluate` on the production machine, ideally with a GPU.
2. **Extend the evaluation set** with 50 or more questions written independently of the tuning set, including questions on the client's own documents, and re-check the 0.40 image threshold.

3. **Pilot ingestion** of a representative batch of client documents with the folder sync and review parsing quality.
4. **Set up monitoring:** schedule daily telemetry logging and drift alerts.
5. **Secure the API** before exposing it beyond localhost: authentication, per-user rate limits and HTTPS through a reverse proxy. The current API has no authentication; uploads are capped at 50 MB per file.

9. Troubleshooting

Symptom	Cause and fix
Web UI shows "API unreachable"	The API is not running or is on another port. Start it (section 4) or set `RAG_API_URL`.
First answer takes several minutes	Models load on first use; expected on CPU.
`IndexCompatibilityError ... --rebuild`	Indexing settings changed. Run `python -m src.cli ingest rebuild`.
`Another ingestion process holds storage/ingest.lock`	Another writer is active (API worker or folder watcher). Stop it; a lock older than 6 hours is cleared automatically.
`AttributeError ... seen_tokens`	Wrong Transformers version. Reinstall `transformers==4.45.2`.
Process killed or out of memory	Two model processes running at once, or less than 16 GB free. Run one at a time.
"New version downloaded" warning from Jina CLIP	Set `HF_HUB_OFFLINE=1` after the first run to keep the cached model code fixed.

Logs are written to `logs/rag.log` (rotating). Each run has a run id, so one session can be traced through the log. Further detail is in `README.md` and the documents in `docs/`.